ΘetaScan: Scan-Parallel Nonlinear Memory

Slow dictionaries, fast nonlinear memories, and associative-scan accumulation

The ThetaScan Project · **Versioned Technical Paper / Public Preview v0.1** · 2026-07-24 · contact: **hi@aim.do**

> **Θ** denotes the parameters of a small memory network. A sequence is represented by causal updates to Θ; *scan* denotes the associative prefix accumulation of those updates.
>
> **Spelling.** The display name **ΘetaScan** uses the Greek capital theta. The ASCII spelling **ThetaScan** is used in filenames, source code, package metadata, and plain-text citations.
>
> **Patent notice.** Core ThetaScan mechanisms described here are the subject of U.S. Provisional Patent Applications No. **64/113,590** and No. **64/118,394**, the second continuing the first. Open directions are research hypotheses, and this notice makes no representation about the scope of any claim. It does not grant a license.
>
> **Status: preliminary.** This is a versioned public preview, not an archival or final paper. The theory is the principal contribution; the experimental section states its own limitations (Section 9.4). Algorithms, terminology, and conclusions may change in later numbered previews.

Abstract

Large language models increasingly mix two kinds of token mixers: exact attention layers and fixed-state recurrent layers, because the recurrent layers keep memory and compute per token constant at long context. Production-scale hybrids such as Jamba, Samba, and Kimi Linear follow this pattern [3, 4, 5]. The quality of such a hybrid depends on what its fixed-state layers can store. This paper studies a family of fixed-state layers built from a small nonlinear network: the network's trained parameters act as a **slow memory** - a dictionary of token directions - and a **fast memory**, accumulated by an associative prefix scan, records where the tokens of the current sequence sit in that dictionary. Every write is computed at the shared slow reference, never at the evolving fast state, so accumulation stays exactly scan-parallel. We describe two write rules (a damped Gauss-Newton step and a normalized Hebbian kernel deposit), the normalized reads that keep retrieval a bounded mixture, a random feature expansion that grows state without growing trainable parameters, and matched multi-timescale temporal views. The empirical question is direct: take an existing training setup, replace attention layers with the proposed memory at the same whole-model parameter count, and check whether the loss stays close - if it does, the layer is replaceable. In a hybrid language-model benchmark run to a completed 7,500-step schedule with terminal warmdown, it does: the best ThetaScan arm finished below the attention control on both raw and quantized validation loss, and every ThetaScan arm finished far ahead of a Mamba-3 control.

1. Introduction

Why study another fixed-state token mixer? Because modern language models already rely on them. Exact self-attention gives every later token a distinct weight on every earlier token [1], but its key-value cache grows with context and its training cost grows quadratically. Fixed-state recurrent mixers - linear attention, fast-weight memories, gated recurrences, and state-space models [2, 6, 7, 8, 9, 10] - keep per-token cost constant. Today's practical answer is the hybrid: most layers are recurrent, and a few attention layers remain. Jamba interleaves Mamba and attention blocks at production scale [3]; Samba shows the same pattern with sliding-window attention [4]; Kimi Linear trains a 48B-parameter hybrid in which a delta-rule linear-attention module carries three of every four layers and reports quality above full attention at much lower decode cost [5]. Every one of these designs asks the same question of its recurrent layer: **how much of the sequence can a fixed-size state usefully hold?**

We use **fixed-state recurrent sequence models** as the umbrella term for this family. Two properties of a member matter most for language modeling:

1. **Token separation.** Nearby tokens - near-duplicate keys with different values - must stay distinguishable inside the state. If the state cannot separate them, a read returns their mixture.

2. **State capacity and cross-talk.** The state has finite size. Every write overlaps earlier writes, and the useful question is how quickly this cross-talk degrades reads as the number of stored records grows.

Compression forces both problems: a fixed-size state cannot keep token-level records, so it must superimpose them. Recall quality is a known separating axis between attention and fixed-state models [11]. The design space of answers is what this paper is about.

ThetaScan's answer is a nonlinear memory with a strict parallelism constraint. The memory is a small network. Its slow, backpropagation-trained parameters decide **which token directions exist and how strongly nearby tokens are pulled apart**. Its fast, per-sequence state records **what the current sequence wrote at those directions**. Every write is computed from the current token and the shared slow parameters - never from the evolving fast state - so writes commute and the whole memory is one associative prefix scan. Nonlinearity enters twice: it shapes the addresses that separate tokens, and it shapes the read.

This paper is organized from the general to the specific. Section 2 reviews the main existing fixed-state designs and the limits that motivate ours. Section 3 defines the slow/fast decomposition and its two write rules. Section 4 gives the concrete nonlinearities and feature maps. Section 5 explains the normalized reads. Section 6 introduces random feature expansion, which grows the state without growing the trainable parameter count. Section 7 adds matched multi-timescale views. Section 8 lists compatible extensions. Section 9 reports the completed 7,500-step benchmark, and Section 10 concludes with future work.

---

2. Existing fixed-state designs and their limits

This section explains why we do not simply reuse an existing write rule. Each design below solves part of the problem; each leaves a gap that the ThetaScan construction targets.

Linear attention. Linear attention replaces the softmax kernel with a factorizable feature map, which turns attention into a recurrence over an outer-product state [2, 12, 13]. Its limit is the address map itself. A linear key map cannot increase the separation of two nearby tokens: distances after a linear map are bounded by the map's norm, so near-duplicate keys stay near-duplicate addresses, and their payloads mix in the state. Capacity grows with key width, but making the key longer under a linear map adds coordinates without adding decorrelation - the new coordinates are linear functions of the same input and inherit the same overlaps. Feature maps that mimic softmax [12] or expand with random features [13, 14] soften this but keep the address geometry fixed rather than learned.

Delta rule and gated variants. Delta-rule memories read the current state before writing, and store the prediction error instead of the raw value [15, 16, 17]. This gives replacement: writing a new value at an old key erases the old value. The price is twofold. First, the write depends on the evolving state, so token transitions stop being a commutative sum. Exact parallel forms exist - modern chunkwise algebra trains the delta rule without approximation [16] - but the algebra works precisely because the state stays *linear* in the stored payloads; it locks the state map, so the delta family cannot pass its state through the kind of nonlinearity this paper is about. Second, erasure is aimed at single-valued recall - key means one value - while language statistics are often multi-valued: a context pattern predicts a distribution of continuations. A rule built to overwrite tends to discard exactly the accumulated evidence that a normalized mixture read would use.

Test-time-training memories. Test-time-training layers and Titans-style memories put an expressive network in the state and run gradient steps on it along the sequence [18, 19]. MesaNet solves a local regression problem per token [20], and test-time regression provides a unifying frame [21]. These are the most expressive fixed-state writes, but the inner optimization is sequential in principle. Practical implementations recover parallelism by chunking: within a chunk, updates are computed against a stale state, so not every past token participates in the linearization that produced the current memory. The parallel form is an approximation to the stated objective, and the approximation error is schedule- and chunk-size-dependent.

The gap, stated positively: we want a memory whose **addresses are learned and nonlinear** (unlike linear attention), whose **writes stay exactly parallel even with a nonlinear state** (the delta rule keeps exactness only by keeping the state linear; test-time rules keep expressivity only by accepting stale chunked gradients), and whose **reads keep the multi-valued evidence** that erasure destroys. That is the regime defined next.

---

3. ThetaScan: slow memory and fast memory

This section defines the family. The purpose of the decomposition is to let backpropagation do what it is good at - building a dictionary - while the scan does what it is good at - accumulating a sequence into a state.

3.1 The decomposition

Consider a residual memory map

$$f_\theta(u) = u + sW_2\,\sigma(W_1 u), \qquad \theta = (W_1, W_2),$$

with a fixed depth scale s and a small numerical stabilizer $\varepsilon > 0$ used throughout. The **slow memory** is $\theta_0 = (W_{1,0}, W_{2,0})$ together with any per-feature controls (thresholds, scales): it is trained by ordinary backpropagation across sequences and is frozen while one sequence is evaluated. The slow memory is the dictionary. Row j of W_1 is a stored direction; the nonlinearity σ decides how sharply a token must align with that direction to activate it; W_2 decides what the feature contributes to the output.

The **fast memory** is the per-sequence state. For projected key, value, and query vectors (k_t, v_t, q_t), every token computes a write from the slow reference only,

$$\Delta\theta_t = \mathcal{W}(\theta_0, k_t, v_t), \qquad \theta_t = \theta_0 + \mathrm{scan}_{i \le t}\,\Delta\theta_i,$$

and the read evaluates the accumulated memory at the query. No $\Delta\theta_t$ reads θ_{t-1}. Additive updates commute, and input-dependent affine decay keeps the transition associative, so the fast memory is exactly one prefix scan - the property that gives the family its name. Gradients from the outer loss flow through all of this normally; what is absent is a sequential inner optimizer.

3.2 Two write rules

Two members of the family are studied. Both deposit at addresses formed by the slow nonlinear dictionary; they differ in what they store.

Gauss-Newton (GN) memory treats the parameters of the memory MLP as the fast state. Each token linearizes f_θ at the slow reference and writes a damped, local Gauss-Newton correction that reduces its own residual $r_t = v_t - f_{\theta_0}(k_t)$. Both W_1 and W_2 receive fast increments; the read evaluates the accumulated increments through the full nonlinear forward. Section 4.1 gives the exact realization.

Normalized kernel memory does not store parameters. It fixes a non-negative feature map ϕ built from the slow dictionary and accumulates sufficient statistics - a payload numerator and a matched feature mass - then reads their ratio. Section 4.2 gives the maps and Section 5 the read.

3.3 Why nonlinear Hebbian accumulation

Both rules are, at bottom, Hebbian: each token adds an outer product of an address and a payload, and the scan is the sum. What the nonlinearity buys is separation. Classical associative-memory theory makes the point directly: with linear (inner-product) addressing, a Hopfield-style memory stores on the order of the key width before cross-talk overwhelms recall, while dense associative memories with higher-order or sharper nonlinear score functions store polynomially or exponentially more patterns, because the nonlinearity suppresses the overlap terms between distinct stored keys [22, 23, 24, 25]. A squared or thresholded activation applied to a normalized score is exactly such a sharpening: two keys with cosine overlap $c < 1$ interfere through $\sigma(c)$, and for convex sharpening σ, $\sigma(c)$ is driven far below $\sigma(1)$. The slow dictionary is trained end to end, so the model can place its directions where the token distribution needs the separation most.

The read is nonlinear as well, and this changes the target behavior. The goal of these memories is **not** unique recall of a single record - the delta rule's target - but the behavior of softmax attention: many stored tokens contribute, with weights that depend nonlinearly and sharply on the query [1, 26]. A normalized read over sharpened non-negative features is precisely a learned, finite-width version of that: a competitive weighting over the sequence's deposits (Section 5). In this sense ThetaScan aims at attention-like mixing at fixed state, not at a key-value store.

One structural note for later work: the output factor W_2 is not essential to the decomposition. The fast state could bind addresses directly to values and dispense with a trainable output factor, the way state-space models carry their values - a W2-less memory. We keep W_2 in v0.1 because the Gauss-Newton write and the two-stage normalized read use it; Section 8 lists the W2-less variant as an open direction.

---

4. Nonlinearities and feature maps

This section specifies the concrete realizations, because the family's behavior depends on the exact nonlinearity and normalization; a reader who wants only the mechanism map can skim to Section 5.

4.1 The GN realization

The idealized single-block derivation linearizes f_θ at θ_0. With $a = W_{1,0}k$, $h = \sigma(a)$, and $J_\sigma = \operatorname{diag}\sigma'(a)$, the Jacobian of the residual branch acts on an increment as

$$J(\delta W_1, \delta W_2) = s\,(\delta W_2 h + W_{2,0} J_\sigma\, \delta W_1 k),$$

its one-token output-space Gram is

$$G = JJ^\top = s^2\left(\|h\|^2 I_d + \|k\|^2\, W_{2,0} J_\sigma^2 W_{2,0}^\top\right),$$

and the damped minimum-norm correction is

$$c = (G + \mu I_d)^{-1} r, \qquad \delta W_2 = s\, c\, h^\top, \qquad \delta W_1 = s\left(J_\sigma W_{2,0}^\top c\right) k^\top.$$

The solve lives in the head output dimension d, not the hidden width m. For later use, name the two write payloads

$$p_i = s J_{\sigma,i} W_{2,0}^\top c_i \in \mathbb{R}^m, \qquad u_i = s^2 c_i \in \mathbb{R}^d.$$

The accumulated read then changes the feature geometry at query time:

$$a_t(q) = W_{1,0} q + \sum_{i \le t} p_i (k_i^\top q), \qquad h_t(q) = \sigma(a_t(q)), \qquad f_t(q) = q + s W_{2,0} h_t(q) + \sum_{i \le t} u_i\,(h_i^\top h_t(q)).$$

This evaluates the accumulated increments exactly; it does not claim a joint nonlinear solve across overlapping writes.

The studied numerical realization always RMS-normalizes each pre-activation. With

$$\nu(a) = \left(\tfrac{1}{m}\|a\|^2 + \varepsilon\right)^{-1/2}, \qquad \widehat{a} = \nu(a) a, \qquad h = \sigma(\widehat{a} - \tau),$$

where τ is a zero-initialized learned per-unit threshold in the thresholded mode and zero otherwise, the exact RMSNorm derivative is

$$D\,\mathrm{RMSNorm}(a) = \nu(a) I_m - \frac{\nu(a)^3}{m}\, a a^\top.$$

The realization deliberately drops the rank-one term and uses the diagonal $\widetilde{J}_h = \nu(a)\operatorname{diag}\sigma'(\widehat{a} - \tau)$ in the hidden write, the Gram, and the dual streams. The honest reading of this approximation runs as follows. The exact Jacobian **suppresses** the radial direction: RMS normalization is scale-invariant along a, so the true derivative of $\widehat{a}$ in the direction of a is zero, and the rank-one term is exactly the correction that removes it. The scalar form νI **keeps** that radial sensitivity - it overestimates the response to perturbations along a and is exact only orthogonally to a. What is discarded is therefore a rank-one *normalizing* correction, not a negligible tail. The compensation is architectural: the write payload is scaled by the same ν, and the read passes through the same RMS-normalized forward, so radial inflation cannot accumulate unboundedly. The practical benefits are unchanged - $\widetilde{J}_h$ stays diagonal, per-token writes cost elementwise work, and the diagonal/Jacobi Gram solve

$$c_j = \frac{r_j}{s^2\left(\|h\|^2 + \|k\|^2 \|(W_{2,0}\widetilde{J}_h)_{j,:}\|^2\right) + \varepsilon}$$

is $O(d)$ per token and well conditioned in low precision. Whether the dropped rank-one correction ever matters at scale is an explicit ablation target.

`jacobian_steps=2` recomputes the residual once at the shifted linearization and writes a second stream; both streams are read together. In SwiGLU mode two pre-vectors are normalized independently and the same scalar-only derivative is applied to each. The public nonlinearities are squared ReLU, thresholded squared ReLU, SiLU, and SwiGLU.

4.2 Kernel feature maps

All kernel maps share two steps: L2-normalize the token, project with a learned slow matrix, RMS-normalize the projection,

$$\bar{x} = \frac{x}{\max(\|x\|, \varepsilon)}, \qquad a_h(x) = \mathrm{RMSNorm}(W_h \bar{x}).$$

Softmax partition. A softmax over learned normalized linear scores,

$$\phi_{h,j}(x) = \frac{\exp(\Gamma_{h,j}[a_{h,j}(x) + b_{h,j}])}{\sum_\ell \exp(\Gamma_{h,\ell}[a_{h,\ell}(x) + b_{h,\ell}])}.$$

Sharpness Γ is fixed, per head, or per feature; this is global competition among learned scores, not a radial map - there are no centers and no distances. Each write deposits unit total mass.

ReLU-squared ridge. Squared rectified scores with an optional learned per-head threshold,

$$\widetilde{\phi}_{h,j}(x) = \left[a_{h,j}(x) - \tau_h\right]_+^2, \qquad \phi_h = \widetilde{\phi}_h / \textstyle\sum_j \widetilde{\phi}_{h,j},$$

with a uniform fallback when every ridge is inactive. Squared ReLU has precedent as a strong decoder nonlinearity [27], and learned thresholds echo shrinkage gates such as JumpReLU [28]. With zero threshold the active region of a feature is a half-space of the normalized score; a learned threshold gates a level set instead.

Projected B-spline. P learned projections each feed a clamped open-uniform cubic B-spline basis of L functions on $[-B, B]$:

$$c_{h,p}(x) = \mathrm{clip}(\Gamma_h\, a_{h,p}(x), -B, B), \qquad \phi_{h,p,j}(x) = \tfrac{1}{P}\, B_j^{(3)}(c_{h,p}(x)).$$

The basis is non-negative and sums to one per projection, and at most four cells are active per projection. The right geometric description is a **projection-structured normalized-score map**: each feature is local along one learned normalized-score coordinate and unconstrained along the others, and the shared RMS normalization couples the projections, so the active set is a band of a normalized score rather than a slab in input space. This construction connects to projection-pursuit regression [29]; classical results on ridge and radial networks supply the approximation background without any claim of unlimited sequence memory [30, 31].

4.3 An observation on sparsity

Across screens, the better-performing address maps shared one property: **few strongly active features per token**. The softmax partition is competitive by construction; the winning ReLU-squared configurations used learned thresholds, which deactivate most ridges per token; the B-spline map activates at most four cells per projection. We report this as a design observation, not a measured law: sparse positive addresses reduce pairwise overlap between stored tokens, which is the cross-talk term of Section 3.3. A quantitative activity/recall curve is future work.

---

5. Normalized reads

Why normalize? Because a fixed-size Hebbian state overloads, and the failure mode should be a **bounded bias, not a growing amplitude**. An unnormalized read grows with the number and strength of matching writes; a normalized read returns a weighted average whose error appears as mixture bias. The bounded form also matches the attention-like goal of Section 3.3: softmax attention outputs a convex combination of values, and a normalized positive-feature read lies in $\mathrm{conv}(\{0, v_1, \ldots, v_t\})$.

5.1 Kernel memory: constitutive normalization

Kernel memory stores, per head (index suppressed),

$$N_t = \sum_{i \le t} \phi(k_i) v_i^\top, \qquad D_t = \sum_{i \le t} \phi(k_i),$$

and reads the ratio

$$\mathcal{R}_t(q) = \frac{\phi(q)^\top N_t}{\phi(q)^\top D_t + \varepsilon} = \sum_{i \le t} \frac{\kappa(q, k_i)}{\sum_{j \le t} \kappa(q, k_j) + \varepsilon}\, v_i, \qquad \kappa(q,k) = \phi(q)^\top \phi(k).$$

The state is a finite-rank causal kernel smoother with no stored token records - a causal Nadaraya-Watson estimator over the learned features. The statistical analogy names the normalization rule only; the features need not be radial or center-based.

5.2 GN memory: one- and two-stage feature mass

For GN the normalization is a read choice with the same non-negative features $\phi_i = [\mathrm{RMSNorm}(W_{1,0} k_i) - \tau]_+^2$ (and ϕ_q for the query). An output-stage read normalizes only the W_2 increments:

$$N_t^{W_2} = \sum_{i \le t} u_i \phi_i^\top, \qquad c_{2,t}(q) = \frac{N_t^{W_2} h_t(q)}{D_t^\top h_t(q) + \varepsilon}.$$

A **two-stage** read also retrieves the hidden correction through a matched statistic before the nonlinearity:

$$N_t^{W_1} = \sum_{i \le t} p_i \phi_i^\top, \qquad c_{1,t}(q) = \frac{N_t^{W_1} \phi_q}{D_t^\top \phi_q + \varepsilon},$$

$$\widetilde{h}_t(q) = \left[\,\mathrm{RMSNorm}(W_{1,0} q + c_{1,t}(q)) - \tau\right]_+^2, \qquad c_{2,t}(q) = \frac{N_t^{W_2} \widetilde{h}_t(q)}{D_t^\top \widetilde{h}_t(q) + \varepsilon}.$$

The write payloads (p_i, u_i) are unchanged and no new trainable parameter appears, but the semantics change: $N_t^{W_1}$ is a positive-feature memory, not the literal $\sum_i \delta W_{1,i}$. Two-stage feature mass is the intermediate regime between accumulated-parameter GN and normalized kernel memory, and it is the read used by the reference benchmark arms. Both normalized GN modes require one Jacobian step and squared-ReLU-family features.

Normalization cannot perform replacement: two incompatible payloads at similar addresses still read as their mixture. It also discards count information that an unnormalized read carries. Normalized and unnormalized reads are therefore distinct hypotheses, and the library exposes both.

---

6. Random feature expansion

This section addresses capacity directly. The cross-talk analysis of Section 3.3 says separation improves with more, sharper features; the state size scales with the feature width m; but the trainable factors $W_1 \in \mathbb{R}^{m\times d}$ and $W_2 \in \mathbb{R}^{d\times m}$ also scale with m. Under a fixed parameter budget, the width of the dictionary competes with the rest of the model. Random feature expansion breaks that coupling: **state and addresses grow; trainable memory parameters do not.**

Store the trainable factors at a base width $b = m/f$ for an integer factor f, and fix two non-trainable sign maps with unit-norm rows,

$$U \in \{\pm 1\}^{m\times b}/\sqrt{b}, \qquad D \in \{\pm 1\}^{b\times m}/\sqrt{m},$$

whose sign patterns are drawn deterministically from a configured key string by an extendable-output hash. The effective memory weights are the compositions

$$W_1^{\text{eff}} = U\,W_1 \in \mathbb{R}^{m\times d}, \qquad W_2^{\text{eff}} = W_2\,D \in \mathbb{R}^{d\times m},$$

and every write and read of Sections 4-5 operates on the effective weights: the GN Jacobian writes against W^{eff}, the ReLU-squared ridge features evaluate through W_1^{eff}, and the fast state, evidence, and per-feature learned thresholds live at the full width m. At $f{=}1$ the mechanism reduces bitwise to the dense memory. The maps are rebuilt from the key at construction and checkpoint load, are excluded from checkpoints, and never train; distinct keys (for example per-layer suffixes) give independent maps.

Why should the expanded features be more than a redundant recoding? The linear part of W_1^{eff} has rank at most b, but each of the m rows is a different random signed mixture of the b trained directions, and each row passes through its **own** nonlinearity with its **own** learned threshold. After sharpening, the m features are not linear functions of each other: the nonlinearity turns rank-b pre-activations into a genuinely m-dimensional positive code, exactly the dimension-lifting role that random features play in kernel approximation [32, 33, 14]. In the Hebbian cross-talk terms of Section 3.3, deposits now overlap through the sharpened m-width code, whose pairwise overlaps are smaller than those of the b-width dense code.

The cost model is explicit: compute and recurrent-state bytes grow with m; the trainable parameter count stays at the base width. The benchmark of Section 9 compares a dense arm and a $2\times$-expanded arm at the **same trainable parameter budget** and reports the state difference openly.

---

7. Matched exponential states and temporal-mode banks

Content addressing alone treats all past tokens equally; language does not. The purpose of the temporal bank is to amplify the contribution of tokens inside an exponential window - adding time-based decorrelation on top of content-based separation - without giving up the undiscounted long view.

An exponentially weighted state follows the associative affine recurrence $A_t = \alpha A_{t-1} + U_t$. For a normalized memory, numerator and denominator must be filtered with **the same** retention,

$$N_t = \alpha N_{t-1} + \phi(k_t)v_t^\top, \qquad D_t = \alpha D_{t-1} + \phi(k_t),$$

otherwise the estimator changes and its magnitude biases. A **temporal-mode bank** keeps the plain sum and J recency views ($\alpha_{j,h} = 2^{-1/H_{j,h}}$ for half-life H), evaluates each view as its own matched ratio, and mixes the completed reads:

$$y_t = \mathcal{R}_t^{(0)}(q_t) + \sum_{j=1}^{J} \eta_{j,h}\left(\mathcal{R}_t^{(j)}(q_t) - \mathcal{R}_t^{(0)}(q_t)\right).$$

Blends η start at zero, so adding a branch never changes the initial function; a bounded variant maps η through $\tanh$. Note the difference from a write-side EMA (`temporal.mode="ema"`): there the state itself decays, so old evidence really disappears. The bank keeps the full sum and only adds recency **views** on top of it - nothing is lost, and each head learns how much recent evidence to add or subtract.

The bank is also a first step toward a general content-by-time code. Each stored feature is effectively $\phi(k_i) \otimes \chi(t-i)$, where the temporal basis χ here contains one constant mode and J exponential modes. A query therefore addresses both *what* was written and *which temporal window* carries it. The static blends above are the simplest read of this code; natural extensions keep the transitions associative while making the temporal side richer - damped oscillatory modes instead of pure decay, or blends that depend on the query, turning the read into a query-conditioned temporal filter bank. The reference arms use the simple form with one or two recency branches.

---

8. Extensions and open directions

Each item here is compatible with the scan-parallel constraint or marks its boundary; none is part of the minimal family definition. We state why each is worth testing.

8.1 Error feedback without giving up the scan

The delta rule reads the current state before writing and stores the prediction error, which gives replacement - at the cost of state-dependent, non-commutative transitions [15, 16]. Several weaker forms of the same feedback idea stay fully scan-parallel, and together they map the boundary:

- **Residual payloads.** Write $v_i - \hat{y}_0(k_i)$ instead of v_i, where $\hat{y}_0$ is the *slow* prediction at the key - the read of the trained prior with no fast state involved. The state then stores only what the slow network cannot already predict, which lowers cross-talk, and the write stays token-local. For the kernel family this prediction already exists as the W_2 prior read, so the variant costs no new parameters.
- **State-corrected addresses.** A second fast state can store feature-space corrections and shift the *query address* at read time, exactly as the two-stage GN read of Section 5.2 does: all corrections are computed from the slow reference in one scan, and the adaptation happens when reading. This is the scan-parallel relative of test-time-training memories [18, 19]: like them, the effective feature map changes along the sequence, but every past token contributes through one exact scan instead of a chunked, stale-gradient approximation.
- **The exact boundary.** With a fixed feature map the state is linear, the true delta transition becomes affine, and exact chunkwise algebra applies [16]. This end of the spectrum remains the clean control: same addresses, Hebbian deposit versus delta replacement, one axis changed.

8.2 Position, gating, and value storage

- **RoPE as token decorrelation.** Input-space rotary embeddings [34] rotate keys and queries before the feature map, making two occurrences of the same token at different positions write at different addresses - position-based decorrelation composing with the content dictionary. Partial input-space RoPE is the benchmark default; a rotation applied *after* a non-negative map would make the mass signed and is rejected for normalized reads.
- **Gating and transition structure from the SSM line.** Input-dependent scalar or per-feature retention - the gating that Mamba-family models refine [8, 9, 17] - transfers directly, since diagonal affine transitions stay associative. Oscillatory or complex modes are the corresponding extension of the temporal bank.
- **W2-less memory.** Bind addresses to values directly and drop the trainable output factor, the way state-space states carry values. This halves the slow memory per feature at equal state and is the natural companion to random expansion: expansion already shows that dictionary quality survives a compressed trainable core.

8.3 Long free-running generation

Recall capacity and generative behavior are different questions, and nonlinear state may help both. An autonomous linear recurrence can only combine its fixed eigenmodes; a nonlinear map can sustain much longer and richer trajectories from the same state size. In an autoregressive loop the generated tokens are written back into the memory, so a nonlinear fast state could support longer and more diverse free-running generation - or it could not; nothing here follows automatically. The two hypotheses - higher associative capacity and richer self-generated dynamics - must be measured separately: recall under controlled load for the first; repetition, cycle length, diversity, and sensitivity to small prompt changes for the second. Neither should be inferred from the other.

9. Empirical evidence

The purpose of the benchmark is a falsifiable minimum: if replacing attention layers with the proposed memory at a matched parameter count tracks the attention control, the approach is applicable in the hybrid regime where fixed-state layers are actually deployed (Section 1). We did not build a bespoke benchmark. We took an existing, fully specified open training project as the host, swapped two of its layers, kept the whole-model trainable parameter count matched, and read the validation loss.

9.1 Protocol

The host is a 9-layer GPT-style decoder from the open Parameter Golf project [35]: width 512, context 1,024, vocabulary 1,024, eight query heads with grouped four-head key/value attention [36]. Two middle mixers (layers 4 and 5) are replaced; everything else is fixed. Each step processes 524,288 FineWeb tokens in bfloat16 on one H100. The **whole-model trainable parameter count** is matched across arms by trimming only the two adjacent FFNs; parameter matching is the control, while state bytes and compute differ. ThetaScan arms use one shared key and query projection with per-head biases [37], partial input-space RoPE, and the `quad` portable evaluator.

Optimizer policy is part of each arm. The attention and Mamba-3 baselines train under the host benchmark's `muon-2d` policy - Muon [38] for two-dimensional matrices, Adam elsewhere. Every ThetaScan arm trains under `muon-2d+theta`, which additionally routes the memory factors W_1, W_2 through batched per-head Muon. The table below therefore compares architecture-plus-policy pairs, not a pure mixer swap under one optimizer; the baselines were not re-tuned with per-head Muon, and a policy-crossed comparison is future work.

The schedule declares 7,500 steps with warmdown over the final 750. The league ran in three exact checkpoint continuations (0→3,000, →4,000, →7,500) on one seed, 1661305741, and **completed the schedule through the warmdown**: the endpoints below are terminally cooled measurements, not mid-schedule snapshots. Raw BPB is validation bits per byte from the training checkpoint; exact-int8 BPB is measured again after an int8 quantization round trip of the weights.

9.2 The five-arm league at step 7,500

arm	trainable parameters	raw bpb @7,500	exact int8 bpb @7,500
GN, 2x random expansion (`gn-expanded-reference-v0.1`)	17,059,928	**1.2327**	**1.23830852**
GN reference, dense (`gn-reference-v0.1`)	17,059,928	1.2342	1.23984702
attention (host control)	17,059,912	1.2349	1.24071718
kernel ReLU-squared ridge, 2x random expansion (`kernel-expanded-reference-v0.1`)	17,059,976	1.2361	1.24215211
Mamba-3 parity control (official module)	17,059,160	1.3194	1.32677632

The replacement goal is met: both GN arms finished below the attention control on both metrics, the expanded kernel arm finished close behind it, and every ThetaScan arm finished far ahead of the Mamba-3 control. The expanded GN arm - the same trainable memory budget as the dense arm at doubled effective width - also improved on its dense counterpart, the capacity effect predicted in Section 6. Limitations are collected in Section 9.4.

9.3 What the measurements suggest

Three suggestions, each with its qualification. First, nonlinear fast memory with normalized two-stage reads is **competitive with attention in the hybrid regime** at matched trainable parameters. Second, **random feature expansion helps at fixed trainable budget**: the expanded GN arm beat the dense arm with the same trainable memory, paying in state; this is the cleanest single-axis observation in the league because the two arms differ only in expansion and its width bookkeeping. Third, the kernel family remains close but behind GN at this scale; whether that is the write rule or the feature map is unresolved - the expanded kernel arm changed both relative to the expanded GN arm. Finally, the GN nonlinearity menu beyond squared ReLU - SiLU and SwiGLU are implemented and exposed - has not been screened at this scale and is an obvious next comparison.

Detailed trajectories, per-arm configuration manifests, parameter contracts, and provenance are in the experiment evidence map and the 7,500-step record.

9.4 Limitations

Single seed; one host model and scale; two swapped layers, so this is evidence for the hybrid regime, not for a pure ThetaScan stack; optimizer policy differs by design between baselines and ThetaScan arms; and the evaluator is a portable materialized path at context 1,024. **No wall-time comparison is reported.** The arms were not run under a matched execution and compilation policy, and the mixer evaluator has since changed, so any step time here would compare implementations and compiler behaviour rather than architectures; a speed claim needs all arms re-measured together under one policy. Long-context recall under controlled load, replication seeds, and a fused scan kernel are the missing pieces for stronger claims.

---

10. Conclusion and future work

A fixed-state token mixer built from a slow nonlinear dictionary and a scan-parallel fast memory trains stably and, in a completed parameter-matched hybrid schedule, finished ahead of its attention control. Two observations stand out. First, every ThetaScan arm finished far ahead of the Mamba-3 control under the same protocol - the family is a serious fixed-state candidate, and continuing this line looks worthwhile. Second, within the family the GN arms lead the kernel arm, and two structural reasons are plausible: the GN state is physically larger (it carries an $m \times m$ address-correction channel that the kernel state lacks), and its read mathematics differs (the kernel's query address is fixed by the dictionary, while GN re-derives the address through the state). Separating these two causes is a concrete next experiment. The mechanisms that carried the result are few: sharpened non-negative addresses, matched normalized reads, one or two recency branches, and - for the strongest arm - random feature expansion at a fixed trainable budget.

Future work follows from the limitations. (1) **Faster execution first:** the current portable evaluator materializes its score matrices; a fused chunked kernel for the whole write-scan-read pipeline, with FLA-based orchestration as the intermediate step, is the highest-leverage engineering item. (2) **Replication:** additional seeds on the same staged protocol. (3) **Other nonlinearities:** SiLU and SwiGLU memory networks are implemented but unscreened. (4) **Long context:** controlled recall under varied record count, similarity, rewrite rate, and query distance, plus length sweeps beyond 1,024. (5) **Optimizer:** a policy-crossed study separating the architecture from per-head Muon [38]. (6) **W2-less memory:** the state-carried-value variant of Section 8, which composes naturally with expansion.

Between linear superposition and sequential test-time optimization lies a regime that is exactly parallel, nonlinear where it matters, and simple enough to audit. Its early numbers justify the systematic study; its ceiling remains a hypothesis.

Reproducibility and disclosure

The accompanying repository provides the reference implementation, scan-versus-materialized parity tests, and the benchmark bootstrap. The completed five-arm league is retained as a compact reviewed record in the 7,500-step continuation set; the evidence map states the source-binding and replay limitations of every numerical claim. Staged runpod configurations for all three continuation stages are checked in. Checkpoint binaries and dataset shards are not redistributed. The expanded arms were measured with fixed expansion maps drawn from the research key namespace; the public `expansion_key` namespace draws statistically equivalent but not bitwise-identical maps, so a new repetition reproduces the protocol rather than the bit pattern.

Research collaboration

The project welcomes collaboration on replication, optimizer and schedule design, fused scan kernels, long-context evaluation, and memory-load testing. Contact **hi@aim.do** before beginning substantial coordinated work.

References

1. A. Vaswani et al. *Attention Is All You Need.* arXiv:1706.03762, 2017.
2. A. Katharopoulos et al. *Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention.* arXiv:2006.16236, 2020.
3. O. Lieber et al. *Jamba: A Hybrid Transformer-Mamba Language Model.* arXiv:2403.19887, 2024.
4. L. Ren et al. *Samba: Simple Hybrid State Space Models for Efficient Unlimited Context Language Modeling.* arXiv:2406.07522, 2024.
5. Kimi Team (Moonshot AI). *Kimi Linear: An Expressive, Efficient Attention Architecture.* arXiv:2510.26692, 2025.
6. J. Schmidhuber. *Learning to Control Fast-Weight Memories.* Neural Computation 4(1), 1992.
7. Y. Sun et al. *Retentive Network: A Successor to Transformer for Large Language Models.* arXiv:2307.08621, 2023.

8. A. Gu, T. Dao. *Mamba: Linear-Time Sequence Modeling with Selective State Spaces.* arXiv:2312.00752, 2023.
9. T. Dao, A. Gu. *Transformers are SSMs (Mamba-2).* arXiv:2405.21060, 2024.
10. A. Lahoti et al. *Mamba-3: Improved Sequence Modeling using State Space Principles.* arXiv:2603.15569, 2026.
11. S. Arora et al. *Zoology: Measuring and Improving Recall in Efficient Language Models.* arXiv:2312.04927, 2023.
12. M. Zhang et al. *The Hedgehog & the Porcupine: Expressive Linear Attentions with Softmax Mimicry.* arXiv:2402.04347, 2024.
13. K. Choromanski et al. *Rethinking Attention with Performers.* arXiv:2009.14794, 2020.
14. H. Peng et al. *Random Feature Attention.* arXiv:2103.02143, 2021.
15. I. Schlag, K. Irie, J. Schmidhuber. *Linear Transformers Are Secretly Fast Weight Programmers.* arXiv:2102.11174, 2021.
16. S. Yang et al. *Parallelizing Linear Transformers with the Delta Rule over Sequence Length.* arXiv:2406.06484, 2024.
17. S. Yang, J. Kautz, A. Hatamizadeh. *Gated Delta Networks: Improving Mamba2 with Delta Rule.* arXiv:2412.06464, 2024.
18. Y. Sun et al. *Learning to (Learn at Test Time): RNNs with Expressive Hidden States.* arXiv:2407.04620, 2024.
19. A. Behrouz et al. *Titans: Learning to Memorize at Test Time.* arXiv:2501.00663, 2025.
20. J. von Oswald et al. *MesaNet: Sequence Modeling by Locally Optimal Test-Time Training.* arXiv:2506.05233, 2025.
21. K. Wang, J. Shi, E. Fox. *Test-Time Regression: A Unifying Framework for Designing Sequence Models with Associative Memory.* arXiv:2501.12352, 2025.
22. D. O. Hebb. *The Organization of Behavior: A Neuropsychological Theory.* Wiley, 1949.
23. J. J. Hopfield. *Neural Networks and Physical Systems with Emergent Collective Computational Abilities.* PNAS 79(8), 2554-2558, 1982.
24. D. Krotov, J. J. Hopfield. *Dense Associative Memory for Pattern Recognition.* NeurIPS, 2016.
25. M. Demircigil et al. *On a Model of Associative Memory with Huge Storage Capacity.* Journal of Statistical Physics, 2017.
26. H. Ramsauer et al. *Hopfield Networks is All You Need.* arXiv:2008.02217, 2020.
27. D. So et al. *Primer: Searching for Efficient Transformers for Language Modeling.* arXiv:2109.08668, 2021.
28. S. Rajamanoharan et al. *Jumping Ahead: Improving Reconstruction Fidelity with JumpReLU Sparse Autoencoders.* arXiv:2407.14435, 2024.
29. J. H. Friedman, W. Stuetzle. *Projection Pursuit Regression.* Journal of the American Statistical Association 76(376), 817-823, 1981.
30. M. Leshno, V. Y. Lin, A. Pinkus, S. Schocken. *Multilayer Feedforward Networks with a Nonpolynomial Activation Function Can Approximate Any Function.* Neural Networks 6(6), 861-867, 1993.
31. J. Park, I. W. Sandberg. *Universal Approximation Using Radial-Basis-Function Networks.* Neural Computation 3(2), 246-257, 1991.
32. A. Rahimi, B. Recht. *Random Features for Large-Scale Kernel Machines.* NeurIPS, 2007.
33. Q. Le, T. Sarlós, A. Smola. *Fastfood: Approximating Kernel Expansions in Loglinear Time.* ICML, 2013; arXiv:1408.3060.
34. J. Su et al. *RoFormer: Enhanced Transformer with Rotary Position Embedding.* arXiv:2104.09864, 2021.
35. OpenAI. *Model Craft Challenge: Parameter Golf.* Software, 2026.
36. J. Ainslie et al. *GQA: Training Generalized Multi-Query Transformer Models.* arXiv:2305.13245, 2023.
37. J.-B. Cordonnier, A. Loukas, M. Jaggi. *Multi-Head Attention: Collaborate Instead of Concatenate.* arXiv:2006.16362, 2020.
38. J. Liu et al. *Muon is Scalable for LLM Training (Moonlight).* arXiv:2502.16982, 2025.